基于 Clang/LLVM 的语言处理流程、SysY 手写实现与 AscendNPU IR 实验

作者：LIZZYGREAT

2026 年 10 月 9 日

摘要

我用一段包含函数、循环、分支和输入输出的 SysY 程序观察 Clang/LLVM 的处理过程，再独立编写 LLVM IR 与 RV64 汇编，并用六组输入核对运行结果。实验中，我对照了预处理文本、LLVM IR、汇编和 ELF 目标文件，也查看了链接后程序如何解析运行库函数。优化部分记录了本例在 x86-64 与 RV64 目标上的不同变化。进阶部分则从源码构建 AscendNPU-IR 的 `bishengir-opt`，按上游测试命令运行 PlanMemory 和 InjectSync，并用上游 FileCheck 断言验证输出。两组 Before/After 均已保存；本次没有安装 CANN，也没有在真实 NPU 上运行程序。通过这些步骤，我重点学习了不同阶段怎样表示同一段程序，以及哪些结论能由实际产物支持。

关键词：编译器；Clang；LLVM IR；RISC-V；ABI；MLIR；lowering

1 引言

刚开始学习编译原理时，预处理、编译、汇编和链接很容易被当成一个连续的“编译”过程。本次作业让我有机会把中间产物分别保存下来，再检查每一步具体做了什么。LLVM IR 让我能追踪源程序中的变量和控制流；RISC-V 汇编则让我观察参数、返回值和局部变量如何使用寄存器与栈帧。

我选择一个短小的求和程序贯穿各阶段：计数器 i 从 1 增加到输入 n，偶数项按 $2i$ 累加，奇数项按 i 累加。它包含局部变量、算术、比较、条件分支、循环、函数调用和运行库输入输出，足以观察作业要求中的主要结构。后续实现使用同一组输入，避免因更换程序而失去对照。

我先观察 Clang 生成的中间文件和目标文件，再对照 LLVM IR 的基本块结构，完成手写 IR 与汇编，最后比较优化结果并记录 AscendNPU IR 的 Pass 实验。正文只保留理解这些结果所需的代码和数据；完整输入、命令与日志放在仓库对应目录中。

2 实验对象与工具环境

2.1 贯穿程序

本次作业使用同一个程序。SysY 源文件 `source/example.sy` 如下。

```
int calc(int n) {
```

```
    int i = 1;
    int sum = 0;
    while (i <= n) {
        if (i % 2 == 0) { sum = sum + i * 2; }
        else            { sum = sum + i;     }
        i = i + 1;
    }
    return sum;
}

int main() {
    int n;      n = getint();
    int result; result = calc(n);
    putint(result);
    return 0;
}
```

程序从 $i = 1$ 开始循环，直到 $i > n$ 后返回累加值。为让 Clang 直接处理，`source/example_clang.c` 保留相同算法，只补充 `getint`/`putint` 的函数声明；宿主机版本由 `source/host_runtime.c` 用标准输入输出实现这两个函数。固定输入为 $1, 2, 3, 4, 5, 10$，预期结果分别为 $1, 5, 8, 16, 21, 85$。这组结果由程序语义先行确定，再用于比较三个实现。

2.2 工具环境

实验环境为 WSL2 上的 Ubuntu 22.04.5。工具版本和可执行文件路径记录在 `logs/00_environment.log`，正文只列出与主要实验步骤直接相关的工具。

表 1: 本实验使用的工具链

工具	版本	本实验用途
Clang / LLVM	14.0.0	预处理、LLVM IR、x86-64 汇编、目标文件；接受手写 `.ll`
GCC / GNU ld	11.4.0 / 系统 GNU ld	系统编译与链接工具
RISC-V unknown-elf GCC	15.1.0	汇编、链接手写 RV64 汇编与课程 runtime
QEMU	7.0.0	用户态运行 RISC-V 可执行文件
Tectonic	0.16.9	编译本报告 LaTeX 源文件

课程目录 `SysY_Compiler_2025` 中已有编译器和 SysY runtime。我只复用其中的 `lib/libsysy_riscv.a`，没有重新构建或修改该目录。Clang 参考汇编与手写程序都使用 `rv64gc/lp64d`，与 runtime 的 ELF 架构标志一致；具体调用和链接命令保存在阶段日志中。

3 Clang 语言处理流程

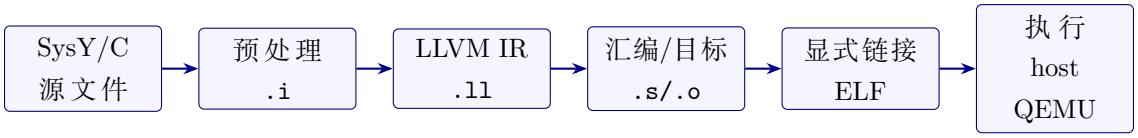


图 1: 从源程序到运行文件的实验流程

3.1 预处理

我运行 `clang -E source/example_clang.c -o generated/example.i` 生成预处理结果。预处理器展开宏、插入头文件、处理条件编译，并在输出中保留源位置标记；它还没有检查程序的类型或控制流。本例没有宏和头文件，`generated/example.i` 的程序语句与输入相同，文件中多出 Clang 写入的行标记。

```
# 1 "source/example_clang.c"
# 1 "<built-in>" 1
# 1 "<built-in>" 3
# 361 "<built-in>" 3
...
# 1 "source/example_clang.c" 2
int getint(void);
void putint(int x);
int calc(int n) {
    int i = 1;
    ...
```

行标记记录源文件与行号，供后续工具定位代码来源；`<built-in>` 表示编译器提供的内建内容。这个例子让我看到，预处理并不一定会显著改变程序文本：没有宏或头文件时，它主要增加位置标记。

3.2 编译器内部各阶段

`clang -cc1` 是真正的编译器前端。对同一份源文件，它依次完成以下工作；本实验通过在 `example.ll` 中看到的结果反推各阶段产物。

表 2: 编译器内部阶段及其在本程序中的表现

阶段	职责	本程序的观察
词法分析	把字符流切成记号	`int`、`calc`、`i`、`<=`、`2` 等记号
语法分析	按文法构造抽象语法树	得到 `while`、`if/else` 的嵌套结构
语义分析	类型检查、建立符号表、插入隐式转换	判定 `i <= n` 为 `i32` 比较，`calc(n)` 类型匹配
IR 生成	遍历 AST 生成 LLVM IR	局部变量变为 `alloca`，语句变为基本块与 `br`
中端优化	在 IR 上做与目标无关的变换	本例 `-O0` 保留栈槽；`-O2` 中出现 SSA 与 `phi`（第 7 节）
后端	指令选择、寄存器分配、指令调度	生成 x86-64 或 RV64 汇编

词法、语法和语义分析通常归入前端；中端处理 IR，后端再面向目标架构。表中的观察来自这一个程序和 Clang 生成的文件，不是对 Clang 内部每个步骤的直接跟踪。

3.3 LLVM IR 生成

我用以下命令输出 LLVM IR：

```
clang -S -emit-llvm -O0 source/example_clang.c -o generated/example.ll
```

命令生成 `generated/example.ll`。在这份 `-O0` IR 中，函数和控制流已显式写出；局部变量通过 `alloca` 取得栈槽，再由 `load`/`store` 读写。第 4 节进一步对照了这些指令与源程序的关系。

3.4 汇编与目标文件

`clang -S -O0` 生成宿主机汇编 `generated/example_x86.s`；`clang -c -O0` 继续调用集成汇编器，将汇编编码为机器码，并输出 ELF64 可重定位文件 `generated/example.o`。汇编器同时记录节区、符号和重定位信息；外部符号的最终地址由链接器确定。对 `example.o` 的检查结果如下。

表 3: `example.o` 的 ELF 特征（`file` 与 `readelf` 实际输出）

检查项	实际结果
文件类型	`ELF 64-bit LSB relocatable, x86-64, not stripped`，即 `ET_REL`
主要节区	`.text`、`.rela.text`、`.eh_frame`、`.symtab`、`.strtab`
已定义符号	`calc`、`main`，类型为 `FUNC`
未定义符号	`getint`、`putint`，均为 `UND`
重定位项	`.rela.text` 中有 `R_X86_64_PLT32` 项，涉及函数调用

`readelf -h` 给出的 ELF 文件头也印证了这一点：

```
Class:                             ELF64
Type:                              REL (Relocatable file)
Machine:                           Advanced Micro Devices X86-64
Entry point address:               0x0
Start of program headers:          0 (bytes into file)
```

Type 为 REL，且该文件没有可执行文件所需的入口布局，因此不能直接作为进程运行。符号表中，calc/main 是本文件定义的函数，getint/putint 则仍需由其他目标文件提供。重定位项记录了调用位置需要链接器补上的地址信息；其中既有运行库函数，也有对 calc 的调用。检查这些内容后，我能区分“汇编已经编码”与“程序已经可以运行”这两个状态。

3.5 链接

本实验先分别生成 example.o 与 host_runtime.o，再用 Clang 驱动显式链接。日志显示驱动调用系统链接器，并加入启动文件和 C 库。链接器把 example.o 中对 getint/putint 的引用与 runtime 中的定义对应起来，再处理重定位并生成可执行文件 generated/reference_linked。它的六组输出与基准一致。单独的 example.o 仍是可重定位文件，也保留未定义的外部符号，因此不能独立运行。

从这些文件可以看到，前处理结果仍是源文本，.ll 显示控制流和运算，.s 面向具体处理器，而 .o 已将指令编码并记录尚待链接器处理的引用。保存每一步的产物后，我能把这些阶段的职责和本程序对应起来。

4 LLVM IR 控制流与手写实现

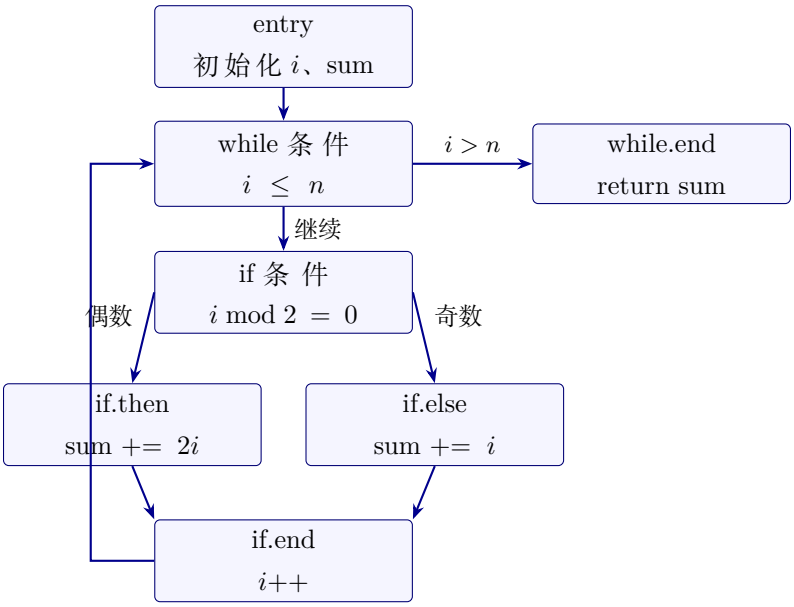


图 2: calc 的基本块与循环回边

Clang -O0 生成的 `calc` 使用数字标签表示基本块。入口初始化 `i` 和 `sum` 后，条件块比较 i 与 n；循环体再判断奇偶并分到两个累加分支。分支合流后递增 `i` 并跳回条件块，形成循环回边。离开循环时，程序读取 `sum` 并返回。阅读 IR 后我发现，源语言的 `while` 和 `if` 并没有对应的专用指令；它们由基本块、比较和 `br` 组合出来。

表 4: 源程序结构与 LLVM 指令对应

源程序结构	LLVM 指令	作用
局部变量声明/赋值	alloca、store	为 n/i/sum 分配栈槽并写初值或新值
读取变量	load	取 i、n、sum 的当前值
加法、乘法、取模	add、mul、srem	计算计数器、累加项与奇偶性
比较	icmp sle、icmp eq	判断循环范围和余数是否为零
if/while	br、基本块	根据条件转移，合并分支并形成循环回边
函数关系	define、call、ret	定义 calc/main，调用 getint/calc/putint 并返回

4.1 手写 LLVM IR

我根据 CFG 独立编写 `llvm/manual.ll`，没有复制 Clang 的 IR。手写版使用 `while.cond` 等语义化块名；形参 `n` 直接作为 SSA 值使用，只有 `i` 和 `sum` 保存在栈槽中。下面只列出循环条件和回边，完整模块保存在 `llvm/manual.ll`。

```
define i32 @calc(i32 %n) {
entry:
  %i.addr = alloca i32, align 4
  %sum.addr = alloca i32, align 4
  store i32 1, i32* %i.addr, align 4      ; i = 1
  store i32 0, i32* %sum.addr, align 4    ; sum = 0
  br label %while.cond

while.cond:
  %i.cond = load i32, i32* %i.addr, align 4
  %continue = icmp sle i32 %i.cond, %n    ; i <= n
  br i1 %continue, label %if.cond, label %while.end

if.cond:
  %i.parity = load i32, i32* %i.addr, align 4
  %remainder = srem i32 %i.parity, 2      ; i % 2
  %is.even = icmp eq i32 %remainder, 0
  br i1 %is.even, label %if.then, label %if.else

  ; 两个累加分支更新 sum 后进入 if.end

if.end:
  %i.current = load i32, i32* %i.addr, align 4
```

```
  %i.next = add nsw i32 %i.current, 1     ; i = i + 1
  store i32 %i.next, i32* %i.addr, align 4
  br label %while.cond                    ; while 的回边

while.end:
  %sum.result = load i32, i32* %sum.addr, align 4
  ret i32 %sum.result
}
```

while.cond 用条件分支决定继续循环还是退出；两个累加分支在 if.end 汇合，随后无条件跳回循环条件。Clang 接受完整的手写 IR，并将它与 host_runtime.c 链接。六组固定输入均通过，完整结果见 outputs/manual-llvm-test.txt。

表 5: 自动生成 IR 与手写 IR 的组织方式对比

对比项	generated/example.ll（Clang -O0）	llvm/manual.ll（手写）
基本块命名	数字标签 %5、%9、%13	语义名 while.cond、if.then、if.end
局部变量	三个 alloca，含形参 n 的栈槽	只给 i、sum 两个 alloca，n 直接用 SSA 形参
分支合流	%13、%18 汇入 %22	if.then、if.else 汇入 if.end
元数据	带 attributes、!llvm.loop、!llvm.module.flags	不产生任何元数据
目标三元组	x86_64-pc-linux-gnu 与完整 datalayout	仅声明 target triple
指令集合	同一组核心指令	逐条对应源语句，无新增或缺失的操作

两份 IR 的核心运算和控制流相同，但块名、元数据以及局部变量的存放方式不同（表 5）。我手写时先按源程序画出控制流，再逐个填写运算和跳转；这样比直接修改 Clang 输出更容易检查语义。

5 RISC-V 汇编与 ABI

5.1 参考汇编与调用约定

我用 Clang 14 交叉生成 RV64 参考汇编（--target=riscv64-unknown-elf -march=rv64gc -mabi=lp64d），先观察后端输出，再与手写实现对照。calc 的序言和奇偶判断片段如下：

```
calc:
    addi sp, sp, -32
    sd   ra, 24(sp)
    sd   s0, 16(sp)
    addi s0, sp, 32
    sw   a0, -20(s0)
```

```
    ...
.LBB0_1:
    lw   a1, -24(s0)      # i
    lw   a0, -20(s0)      # n
    blt  a0, a1, .LBB0_6
    ...
.LBB0_2:
    lwu  a0, -24(s0)      # i % 2 的替代实现
    srli a1, a0, 31
    add  a1, a1, a0
    andi a1, a1, -2
    subw a0, a0, a1
    bne  a0, zero, .LBB0_4
```

与手写版本相比，这次 Clang 输出的 `calc` 与手写版都使用 32 字节栈帧，`ra`、`s0` 和局部变量的位置也相同。两版对奇偶性的处理不同：Clang 把已知除数为 2 的余数计算化成一组移位和算术指令；手写版直接用 `remw`。`rv64gc` 包含整数乘除扩展，因此 `remw` 是可用指令，Clang 选择算术展开并非因为架构没有取余操作。这个例子中 `i` 从正数 1 开始递增，所以该变换与程序使用范围相符。两版的 `main` 栈帧也不同：Clang -O0 使用 48 字节，手写版使用 16 字节，反映出未优化代码和人工组织值的方式不同；这不代表某一版运行更快。

表 6: 本实验用到的 RISC-V 寄存器与调用约定

寄存器/规则	本实验用法
`a0`	第一个整型参数与整型返回值；getint、calc、putint 之间传值
`t0-t2`	calc 内的 caller-saved 临时值；本实验只用这三个
`s0`	calc 的 frame pointer；属被调用者保存，成对保存/恢复
`sp`	栈指针；calc/main 分别保留 32/16 字节且保持 16 字节对齐
`ra`	返回地址；calc 与 main 跨越函数调用保存/恢复

5.2 栈帧布局

RV64 调用约定要求 `sp` 16 字节对齐。`calc` 的栈帧共 32 字节：顶部保存返回地址与旧的 `s0`，中部放三个局部变量。堆栈从高地址向低地址增长，因此“入栈”即减小 `sp`。

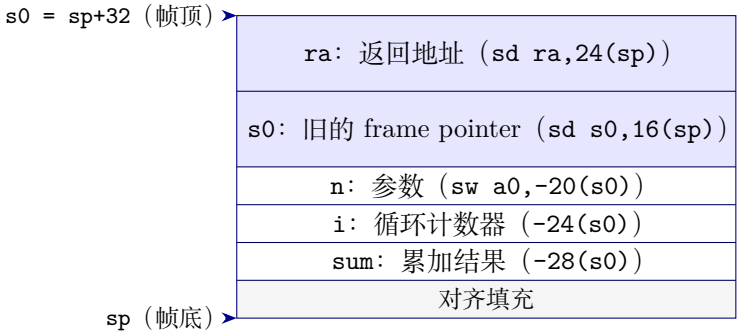


图 3: calc 的 32 字节栈帧（addi sp,sp,-32）

5.3 手写汇编

手写 riscv/manual.s 独立遵守同一约定：序言建立栈帧并保存 ra/s0，循环条件用 blt 在 i > n 时退出，奇偶判断用 remw 加 beqz，偶数项用 slliw 左移一位代替乘 2，累加用 addw，循环尾递增 i 后无条件跳回条件块。控制流标签与第 4 节的 LLVM 基本块一一对应。

```
calc:
    addi sp, sp, -32
    sd   ra, 24(sp)
    sd   s0, 16(sp)
    addi s0, sp, 32
    sw   a0, -20(s0)       # 保存参数 n
    ...
.Lwhile_cond:
    lw   t0, -24(s0)       # i
    lw   t1, -20(s0)       # n
    blt  t1, t0, .Lwhile_end
.Lif_cond:
    li   t1, 2
    remw t2, t0, t1
    beqz t2, .Lif_then
    ...
.Lwhile_end:
    lw   a0, -28(s0)       # 返回值
    ld   s0, 16(sp)
    ld   ra, 24(sp)
    addi sp, sp, 32
    ret
```

完整实现见 riscv/manual.s。其中 main 用 16 字节栈帧保存 ra，依次调用 getint、calc、putint；整数参数和返回值都经 a0 传递。

5.4 链接与运行

手写汇编由 `riscv64-unknown-elf-gcc` 汇编，并静态链接课程已有的 SysY runtime 归档 `libsysy_riscv.a`。关键链接选项沿用课程配置：

```
-static -mcmodel=medany -Wl,--no-relax,-Ttext=0x90000000
```

产物 `generated/manual_riscv` 是 RISC-V ELF 可执行文件，入口地址为 `0x90000044`，由 `qemu-riscv64` 用户态模拟执行。runtime 的析构函数会向 stderr 打印 `TOTAL: 0H-0M-0S-0us` 计时摘要；脚本只比较标准输出，因此该摘要不参与数值判定。

6 运行验证

表 7: 六组输入在三种实现上的输出比较

输入 n	Reference	手写 LLVM	手写 RISC-V/QEMU	结果
1	1	1	1	PASS
2	5	5	5	PASS
3	8	8	8	PASS
4	16	16	16	PASS
5	21	21	21	PASS
10	85	85	85	PASS

表 7 的数字来自脚本 `scripts/09\_final\_validation.sh` 的实际输出。测试框架 `source/test\_reference.sh` 将六组输入逐一传给被测程序，并与事先确定的期望值比较，而不是把三个实现彼此作为基准。这样，即使三个实现以相同方式出错，也不会仅因输出一致而通过。

参考程序和两个手写实现的单次结果保存在 `outputs/` 下，汇总表为 `final-validation.md`，完整命令输出见 `logs/09_final_validation.log`。三种程序都与预期结果一致。固定测试不能覆盖所有输入，但提供了检查本例手写 IR 和汇编的明确对照。

7 优化观察：-O0 与 -O2

我用同一份 C 文件生成两种优化级别的 LLVM IR、x86-64 汇编和 RV64 汇编，再比较实际 diff。下面的观察只针对这个程序和这两种目标。

表 8: LLVM IR 指令数量与结构变化

指标	-O0	-O2
`alloca`	6	0
`load`	11	0
`store`	9	0
`phi`	0	32
局部变量表示	栈槽 + 内存读写	SSA 临时值 + 循环 `phi`
奇偶判断	`srem` + `icmp eq` + `br`	`and`/`xor`/`shl` 位运算
`main` 对 `calc` 的调用	存在 `call @calc`	已内联；`@calc` 定义因外部可见而保留

7.1 从内存槽到 SSA

在这份 `-O0` IR 中，局部变量通过栈槽和 `load`/`store` 访问；`-O2` 结果则把这些变量保存在 SSA 值中，并用 `phi` 传递循环迭代间的值。统计结果为 `alloca`/`load`/`store` 从 6/11/9 降到 0/0/0，`phi` 从 0 增至 32。对我来说，最明显的变化不是源代码语句消失，而是同一状态不再通过内存读写表达。

7.2 奇偶判断被改写成位运算

源程序用 `i % 2 == 0` 选择分支。`-O2` 把奇偶判断从取模与分支改写为位运算。RV64 的 `calc` 中，奇偶位决定左移 0 位还是 1 位，因而两条累加路径可以合成一条：

```
calc:
    li   a1, 0
    blez a0, .LBB0_3
    li   a2, 1
.LBB0_2:
    mv   a3, a2
    not  a2, a2
    andi a2, a2, 1
    sllw a2, a3, a2        # i << (i 为偶数 ? 1 : 0)
    addw a1, a1, a2        # sum += 该项
    addiw a2, a3, 1
    blt  a3, a0, .LBB0_2
.LBB0_3:
    mv   a0, a1
    ret
```

`not`/`andi` 生成移位量，`sllw` 再按该值左移：偶数项乘 2，奇数项保持原值。这样，输出汇编不再保留原来的 `if`/`else`。优化后的 `calc` 也不再需要栈帧，因为它不调用其他函数且只使用临时寄存器。

7.3 两种目标的低层差异

同一份较高层的优化结果，最终仍要由目标后端决定怎样使用本机指令。

x86-64 的结果中可以看到四个 `i32` 组成的向量运算、XMM 寄存器和 `paddd`；循环末尾还保留处理不足四个元素的路径。RV64 目标为 `rv64gc`，不含 RVV 扩展，所以输出仍是标量循环，没有 RVV 向量指令。两种目标都把 `calc` 内联进 `main`，但向量化结果不同。

两种目标生成的汇编长度变化方向不同：x86-64 因向量循环和内联而增加，RV64 因控制流与栈访问减少而变短。汇编文本行数不能替代运行时间、代码体积或能耗测量；本实验没有测量这些指标，因此不据此推断性能提升。

8 MLIR 与 AscendNPU IR

前几节中，我看到 Clang 的 LLVM IR 已经显式表示变量读写和分支；但面向 NPU 时，还需要表达张量形状、存储位置、数据搬运和硬件流水线。MLIR 允许编译器用不同 Dialect 保留这些层次的信息，再逐步 lowering。较高层的张量与算子语义有利于融合和 tiling；HIVM 则能表达 GM、UB 以及对应的数据搬运和向量运算。图 4 是依据课程要求与官方资料整理的概念路线，并非本机完整运行得到的转换链。[1, 2, 4]

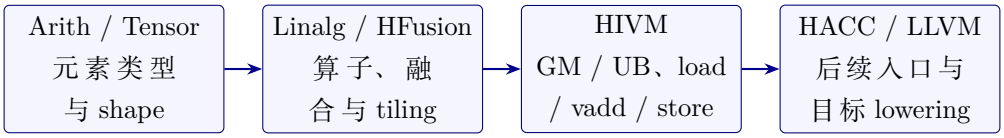


图 4: VecAdd 的概念 lowering 路线；本次实跑从 HIVM 层开始

这条概念路线中，高层表示主要描述“两个向量逐元素相加”；到了 HIVM，内存空间和搬运步骤已显式出现。官方 VecAdd 样例使用 16 个 `i16` 元素：A、B、C 位于 GM，输入通过 `hivm.hir.load` 搬入 UB，在 UB 上执行 `hivm.hir.vadd`，再由 `hivm.hir.store` 写回 GM。下列代码是对上游文件的结构摘录，省略了完整类型和入口属性；它不是我从 Arith/Tensor 实际 lowering 得到的输出。[4, 5]

```
func.func @add(%a: memref<16xi16, #hivm.address_space<gm>>,
               %b: memref<16xi16, #hivm.address_space<gm>>,
               %c: memref<16xi16, #hivm.address_space<gm>>) {
  %ua = memref.alloc() : memref<16xi16, #hivm.address_space<ub>>
  hivm.hir.load ins(%a : ...) outs(%ua : ...) // GM -> UB
  %ub = memref.alloc() : memref<16xi16, #hivm.address_space<ub>>
  hivm.hir.load ins(%b : ...) outs(%ub : ...) // GM -> UB
  %uc = memref.alloc() : memref<16xi16, #hivm.address_space<ub>>
  hivm.hir.vadd ins(%ua, %ub : ...) outs(%uc : ...)
  hivm.hir.store ins(%uc : ...) outs(%c : ...) // UB -> GM
  return
}
```

代码摘录保留了本例的关键操作。memref.alloc 标出 UB 临时区，load/store 标出 GM 与 UB 之间的数据移动；算术本身由 vadd 表达。硬件相关信息进入 IR 后，后续 Pass 才能针对这些存储和执行约束继续处理。

8.1 实测一：PlanMemory 规划 UB 地址

PlanMemory 根据临时 buffer 的生命周期和目标硬件约束安排片上地址；在这个用例中，它把 memref.alloc 改写为带偏移的 hivm.hir.pointer_cast。我选用上游 plan-memory.mlir 中的 test_mem_allocate_basic：其中四个 UB buffer 都是 $16 \times 16 \times 16$ 的 f16 张量，每个占 8192 字节。测试自带的默认策略命令为：

```
// RUN: bishengir-opt %s -hacc-append-device-spec=target=Ascend910B1 \
//        -hivm-plan-memory -split-input-file -verify-diagnostics | FileCheck %s
// CHECK-LABEL: func.func @test_mem_allocate_basic
// CHECK-NOT: memref.alloc()
// CHECK: arith.constant 0 : i64
// CHECK: hivm.hir.pointer_cast
```

-hacc-append-device-spec 先把 Ascend910B1 的目标规格写入模块，随后运行 PlanMemory。测试文件还包含一条使用 largest-first 策略的 RUN。我逐字执行这两条命令，并将结果交给上游 FileCheck 断言检查；两条均通过。

对默认策略用例保存的 Before/After 显示：

表 9: PlanMemory 在该用例上的实测变化

指令计数	Before	After
memref.alloc()	4	0
hivm.hir.pointer_cast	0	4
hivm.hir.load/vadd/store	5	5

输出中四个分配都成为 pointer_cast(\%c0_i64)，即偏移为 0；load、三次 vadd 和 store 保持不变。模块里还出现 UB 容量、对齐等目标参数。需要区分的是，这些参数由命令中的 -hacc-append-device-spec 加入，不是 PlanMemory 自己生成的。该用例的 offset-0 结果只描述这组输入和策略，不能推广到其他程序；更复杂的生命周期和别名关系可能得到不同布局。Before/After 文件保存在 mlir/outputs/。[7, 8, 9]

8.2 实测二：InjectSync 表达流水线依赖

第二个上游用例 test_mem_injct_sync_basic 只有 load、vadd 和 store 三个操作，输入中没有同步指令。运行测试中的 -hivm-inject-sync -hivm-lower-multi-buffer-counter 后，IR 增加了以下依赖关系：

```
hivm.hir.load ins(%arg0 : ...<gm>) outs(%0 : ...<ub>)
hivm.hir.set_flag[<PIPE_MTE2>, <PIPE_V>, <EVENT_ID0>]
```

```
hivm.hir.wait_flag[<PIPE_MTE2>, <PIPE_V>, <EVENT_ID0>]
hivm.hir.vadd ins(%0, %2 : ...) outs(%0 : ...)
hivm.hir.set_flag[<PIPE_V>, <PIPE_MTE3>, <EVENT_ID0>]
hivm.hir.wait_flag[<PIPE_V>, <PIPE_MTE3>, <EVENT_ID0>]
hivm.hir.store ins(%0 : ...<ub>) outs(%arg1 : ...<gm>)
hivm.hir.pipe_barrier[<PIPE_ALL>]
```

MTE2 与 MTE3 表示数据搬运流水，V 表示向量计算流水。Pass 在 load 后加入 MTE2 到 V 的 flag/wait，在 store 前加入 V 到 MTE3 的 flag/wait，并在函数末尾加入一条 `PIPE\_ALL` barrier。统计结果是 2 条 `set\_flag`、2 条 `wait\_flag` 和 1 条 `pipe\_barrier`；Before 文件中的真实指令数为 0。这个输出让我看到，IR 中的操作顺序与不同硬件流水线之间的完成依赖是两件事，Pass 会将需要等待的依赖显式写出来。Before/After 文件保存在 `mlir/outputs/`。[10, 8]

8.3 构建与验证方式

我在用户级虚拟环境中安装了 CMake 4.4.4 和 Ninja 1.13.2，没有替换系统工具。AscendNPU-IR 与 LLVM 子项目分别固定在提交 `2ac1b376` 和 `06913395`；构建出的工具报告版本为：

```
bishengir-opt 2.0.0 (gitcode.com/Ascend/ascendnpu-ir 2ac1b376da1d)
llvm 19.1.7 06913395ef8a   Release build
```

构建工具时遇到上游静态库之间的循环依赖。我在本地补丁中使用 CMake 的 `LINK\_GROUP:RESCAN` 让链接器重新扫描相关静态库；补丁只改变 `bishengir-opt` 的链接参数，没有修改 Pass 实现。详细构建过程和补丁见 `records/12\_ascend\_build.md` 与 `scripts/patches/`。随后三条上游 RUN 命令全部通过 FileCheck，包括 PlanMemory 的两种策略和 InjectSync。

8.4 边界

这部分实验止于 IR Pass。Arith/Tensor 与 HFusion 是我根据资料整理的前置层次，本机实跑的输入从 HIVM 开始；我验证了 Pass 对 IR 的结构变换，但没有验证最终设备上的运行表现。本次没有安装 CANN、配置驱动、生成 kernel `.o` 或在 NPU 上运行。[11, 6]

9 实验可复现性

我把主要命令整理为 scripts/ 下的脚本，并将输出和过程记录分别放在 logs/ 与 records/。自动生成文件保存在 generated/，手写 LLVM IR 与汇编分别保存在 llvm/ 和 riscv/，便于区分工具输出与人工实现。

表 10: 各阶段复现入口

脚本	作用
00_environment_check.sh	记录系统与工具版本，作为环境快照
01_reference.sh	编译宿主机 Ground Truth 并跑六组测试
02_pipeline.sh	生成 .i / .ll / .s / .o 并显式链接
03_llvm_analysis.sh、03_llvm_validation.sh	抽取 IR 做 CFG 分析；验证手写 manual.ll
04_riscv_reference.sh、05_riscv_validation.sh	生成 RV64 参考汇编；汇编链接手写 manual.s 并在 QEMU 运行
06_optimization.sh	生成 O0/O2 三种目标的对照与 diff
09_final_validation.sh	三路实现统一比对并输出总表
10_build_report.sh	用 Tectonic 编译本报告
11_ascend_configure.sh	配置构建目录并确认 bishengir-opt target
12_ascend_build.sh	构建 bishengir-opt
13_ascend_pass_runs.sh	运行三个上游 Pass 测试并保存 Before/After

前 10 个脚本使用课程环境中已有的工具。Ascend 的配置、构建和 Pass 测试还依赖仓库外的源码、LLVM 子项目、虚拟环境及本地链接补丁；这些资源的版本和获取过程记录在 records/11_ascend_build_precheck.md 与 records/12_ascend_build.md 中，未包含在本仓库。运行 RISC-V 阶段前需加载 ../env.sh；各脚本的输入、命令和实际结果可与对应日志核对。

10 结论

这次实验让我把源程序、LLVM IR、汇编、目标文件和可执行程序联系起来。检查 example.o 时，我看到它仍是 ET_REL 文件，并保留对 getint/putint 的引用；显式链接之后，这些引用才由 runtime 提供定义。这个对照比单看命令更容易让我分清汇编器和链接器的职责。

在 LLVM IR 中，我将循环条件、奇偶分支和回边对应到基本块与 br 指令，再独立写出手写 IR。RV64 实验则让我从另一个层面看到同一控制流：参数和返回值使用 a0，calc 通过栈帧保存需要跨指令保留的状态。手写版本与 Clang 输出在栈帧布局上接近，但对奇偶判断的指令选择不同。

优化实验显示，`-O2` 不只是删掉指令：它先改变变量表示，再化简分支和算术；最终是否向量化还取决于目标架构。本例在 x86-64 上出现向量运算，在不含 RVV 的 RV64 目标上仍是标量循环。由于没有测量运行时间，我只能比较代码结构，不能据此判断实际快慢。

AscendNPU IR 部分让我看到硬件限制如何进入较低层表示。PlanMemory 用例把四个临时分配转换为四个偏移为 0 的 `pointer\_cast`，计算操作保持不变；InjectSync 则在搬运与向量计算之间加入 flag/wait，并在函数末尾加入 barrier。三条上游 RUN 命令及其 FileCheck 断言都通过。Arith/Tensor 到 HFusion 的部分仍是资料整理，实测范围从 HIVM 开始；本次没有编译 kernel 或在 NPU 上运行。

整体上，我完成了作业要求的处理流程观察、手写实现和固定用例验证，也从真实 Pass 输出中补上了一个小型的 Ascend IR 实验。还有不少编译器实现细节没有涉及；对这次作业而言，我目前能用保存的产物解释的是各阶段表示的变化及其边界。

参考文献

[1] LLVM Project. *MLIR Language Reference*. https://mlir.llvm.org/docs/LangRef/（访问日期：2026-10-07）.

[2] LLVM Project. *MLIR Rationale*. https://github.com/llvm/llvm-project/blob/main/mlir/docs/Rationale/Rationale.md（访问日期：2026-10-07）.

[3] LLVM Project. *Targeting LLVM IR from MLIR*. https://mlir.llvm.org/docs/TargetLLVMIR/（访问日期：2026-10-07）.

[4] Ascend. *AscendNPU-IR Framework Integration*. https://github.com/Ascend/AscendNPU-IR/blob/master/docs/source/en/developer_guide/conversion/framework_interface.md（访问日期：2026-10-07）.

[5] Ascend. *HIVM VecAdd add.mlir*. https://github.com/Ascend/AscendNPU-IR/blob/master/bishengir/test/Integration/HIVM/VecAdd/add.mlir（访问日期：2026-10-07）.

[6] Ascend. *HIVM VecAdd README and host wrapper*. https://github.com/Ascend/AscendNPU-IR/blob/master/bishengir/test/Integration/HIVM/VecAdd/README.md（访问日期：2026-10-07）.

[7] Ascend. *Plan Memory*. https://ascendnpu-ir.gitcode.com/en/developer_guide/features/PlanMemory/PlanMemory.html（访问日期：2026-10-07）.

[8] Ascend. *HIVM Passes*. https://ascendnpu-ir.gitcode.com/en/developer_guide/passes/hivm_passes.html（访问日期：2026-10-07）.

[9] Ascend. *PlanMemory Test Case*. https://github.com/Ascend/AscendNPU-IR/blob/master/bishengir/test/Dialect/HIVM/plan-memory.mlir（访问日期：2026-10-09）.

[10] Ascend. *InjectSync Test Case*. https://github.com/Ascend/AscendNPU-IR/blob/master/bishengir/test/Dialect/HIVM/inject-sync.mlir（访问日期：2026-10-09）.

[11] Ascend. *AscendNPU-IR Build and Installation*. https://github.com/Ascend/AscendNPU-IR/blob/master/docs/source/en/introduction/quick_start/installing_guide.md（访问日期：2026-10-07）.

[12] Compiler-Labs. 编译原理作业要求与执行流程（本地课程文档，访问日期：2026-10-07）.